arXiv:submit/7371426 [cs.CL] 16 Mar 2026

ByteDance | Seed

Mixture-of-Depths Attention

Lianghui Zhu[1,2,*], **Yuxin Fang**[2,†], **Bencheng Liao**[1,2,*], **Shijie Wang**[2], **Tianheng Cheng**[2], **Zilong Huang**[2], **Chen Chen**[2], **Lai Wei**[2], **Yutao Zeng**[2], **Ya Wang**[2], **Yi Lin**[2], **Yu Li**[2], **Xinggang Wang**[1,✉]

[1]School of EIC, Huazhong University of Science & Technology, [2]ByteDance Seed

[†]Project Lead, ✉Corresponding author

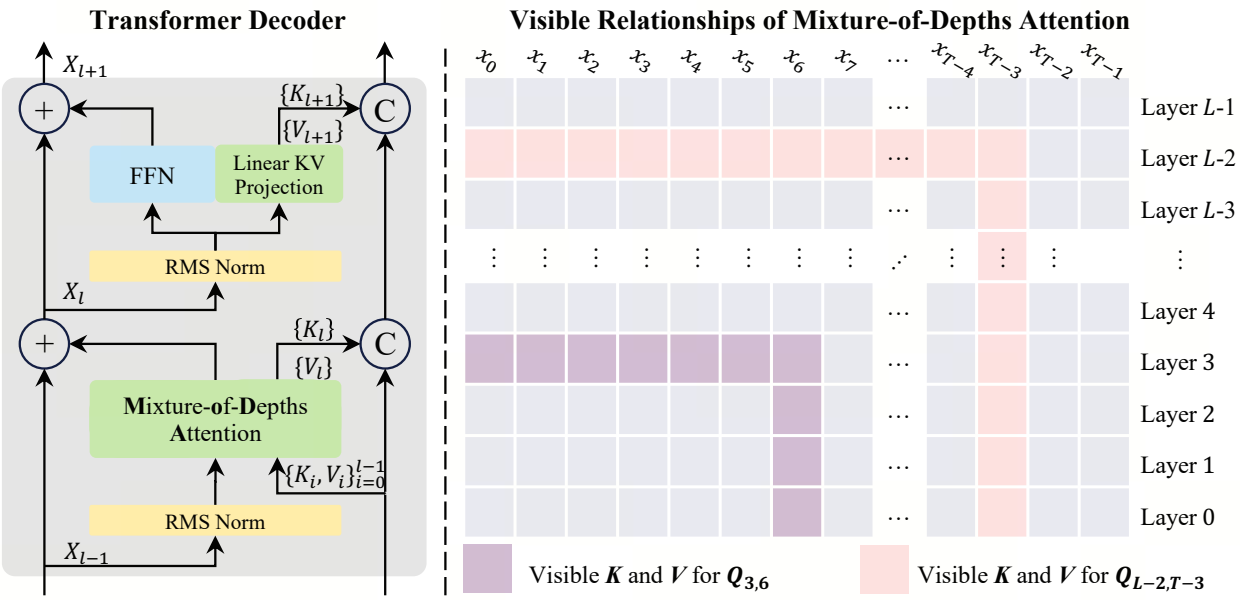

Figure 1 We propose mixture-of-depths attention (MoDA) to address the modern LLM's information dilution problem in a dynamic and hardware-efficient way. Compared with vanilla causal sequence attention, MoDA additionally allows query to attend to depth memories, i.e., depth KV pairs $\{K_i, V_i\}_{i=0}^{l-1}$ at the same query position from preceding layers.

Abstract: Scaling depth is a key driver for large language models (LLMs). Yet, as LLMs become deeper, they often suffer from *signal degradation*: informative features formed in shallow layers are gradually diluted by repeated residual updates, making them harder to recover in deeper layers. We introduce *mixture-of-depths attention* (MoDA), a mechanism that allows each attention head to attend to sequence KV pairs at the current layer and depth KV pairs from preceding layers. We further describe a hardware-efficient algorithm for MoDA that resolves non-contiguous memory-access patterns, achieving 97.3% of FlashAttention-2's efficiency at a sequence length of 64K. Experiments on 1.5B-parameter models demonstrate that MoDA consistently outperforms strong baselines. Notably, it improves average perplexity by 0.2 across 10 validation benchmarks and increases average performance by 2.11% on 10 downstream tasks, with a negligible 3.7% FLOPs computational overhead. We also find that combining MoDA with post-norm yields better performance than using it with pre-norm. These results suggest that MoDA is a promising primitive for depth scaling.

Correspondence: xgwang@hust.edu.cn

Code: https://github.com/hustvl/MoDA

[*]This work was done when Lianghui Zhu and Bencheng Liao was interning at ByteDance Seed.

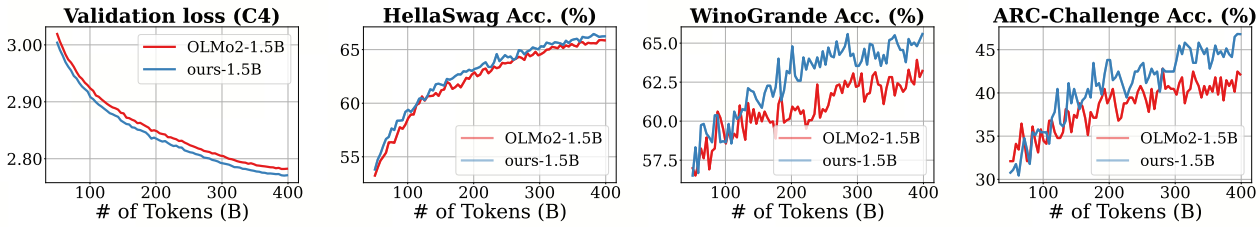

Figure 2 Comparing MoDA and strong open-sourced baseline, i.e., OLMo2 [27], with validation loss and downstream performance under the 1.5B-parameter setting. Models using MoDA achieve lower C4 [30] validation loss and better downstream performance, i.e., HellaSwag [48], WinoGrande [32], and ARC-Challenge [10], than OLMo2.

1 Introduction

Recent progress in large language models (LLMs) [1, 15, 23, 37] has been driven by scaling along four major dimensions: context length [8, 11, 47], training data [1, 37], model width [4, 38], and model depth [6, 40]. Although these dimensions remain effective, incremental gains are becoming increasingly costly, motivating interest in complementary architectural scaling strategies. In current LLM practice, scaling is often realized more through data, context, and especially width, whose optimization behavior and system efficiency are generally easier to realize at scale. Depth, by contrast, remains comparatively under-exploited despite its strong representational appeal. In principle, deeper stacks can support richer hierarchical computation. Yet modern Transformers often fail to convert additional layers into proportional benefits due to the optimization problem [16] and information dilution [20, 28]. The resulting question is central to the architecture design: how can a model scale depth while maintaining optimization stability and preventing information dilution?

The standard residual pathway (ResNet-style) improves optimization stability in deep networks [16], but it still compresses depth history into a single hidden-state trajectory, leaving information dilution largely unresolved. Many methods [22, 42, 49] have been tried to address this problem by upgrading the residual connection. Dense cross-layer connections (DenseNet-style) preserve richer layer-wise history and thus mitigate information dilution [7, 20, 28], but their parameter growth is substantial at LLM scale, which has limited their adoption as a mainstream architecture. The success of attention [39] in sequence modeling suggests a broader principle: data-dependent dynamic mixing can preserve and retrieve historical information more effectively than fixed-pattern aggregation. This motivates extending the same principle from sequence modeling to depth modeling, i.e., enabling each layer to adaptively read useful states from earlier layers. Adaptive cross-layer retrieval is therefore promising, yet practical designs still require a better balance among expressivity, efficiency, and hardware friendliness.

In this work, we introduce *mixture-of-depths attention* (MoDA), a unified attention mechanism in which each head jointly attends to sequence KV of the current layer and depth KV from all preceding layers. Methodologically, we analyze Transformer stacking through a "read, operate, write" lens, comparing depth residual, depth dense, and depth attention in a common design space. MoDA occupies an efficient point that preserves data-dependent depth retrieval without dense cross-layer overhead.

To make MoDA practical at scale, we develop a hardware-aware implementation [12, 13, 43] that fuses sequence and depth attention in one forward pass with shared online-softmax states. Besides, the proposed chunk-aware depth-KV layout and group-aware indexing significantly improve memory access efficiency. This fused kernel reaches 97.3% of FlashAttention-2 efficiency at 64K sequence length, showing that depth-aware aggregating can be integrated without sacrificing modern GPU efficiency.

We validate MoDA on decoder-only language models trained with the 400B-token OLMo2 recipe [27] at 700M and 1.5B scales. In our main 1.5B setting, MoDA improves average perplexity by 0.2 across 10 validation benchmarks and increases average downstream performance by 2.11% on 10 tasks. We also find that combining MoDA with post-norm yields better performance than using it with pre-norm. Additional analyzes, i.e., model-size scaling, attention visualization, and layer-number studies, show robust gains and reduced attention-sink [41] behavior via better probability allocation to informative sequence and depth KV.

The contributions of this paper are summarized as:

- We propose MoDA, a unified attention formulation for dynamic mixtures of sequence and depth, which improves the aggregation of depth-wise information and addresses the information dilution problem of modern LLMs in a data-dependent way.
- We present a hardware-efficient fused algorithm that makes MoDA practical for long-context LLM training. It reaches 97.3% of FlashAttention-2 efficiency at 64K sequence length with numerical precision within the allowed range.
- We provide extensive empirical evidence and comprehensive ablations that MoDA consistently and substantially outperforms the strong open-source baseline, OLMo2, across large-scale corpora at multiple model scales, validating each design choice and establishing MoDA as a reliable foundation for depth scaling in LLMs.

2 Mixture-of-Depths Attention

2.1 Preliminary

Most modern large language models are built on the Transformer architecture [39], where self-attention is the primary token-mixing operator. Given a sequence of T tokens $X = (x_1, x_2, \ldots, x_T) \in \mathbb{R}^{T\times D}$ (with hidden dimension D), self-attention first projects tokens into queries (Q), keys (K), and values (V) via trainable matrices $W_Q \in \mathbb{R}^{D\times(H_q d)}$ and $W_K, W_V \in \mathbb{R}^{D\times(H_k d)}$. Under grouped query attention (GQA) [2], $H_q = GH_k$, $H_k = H_v$, and $D = H_q d$:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{1}$$

where $Q \in \mathbb{R}^{T\times(H_q d)}$ and $K, V \in \mathbb{R}^{T\times(H_k d)}$. The attention operator computes pairwise similarity between queries and keys, applies a softmax to obtain per-head attention weights $A_h \in \mathbb{R}^{T\times T}$, and returns a weighted sum of values:

$$\text{Attention}(Q, K, V) = \text{Concat}_{h=1}^{H_q}\left(\text{softmax}\left(\frac{Q_h K_{\phi(h)}^T}{\sqrt{d}} + \mathcal{M}\right) V_{\phi(h)}\right) \tag{2}$$

where $Q_h \in \mathbb{R}^{T\times d}$, $K_j, V_j \in \mathbb{R}^{T\times d}$, and $\phi(h) = \lceil h/G \rceil$ maps each query head to its shared key-value head. Here, $\mathcal{M} \in \mathbb{R}^{T\times T}$ is an additive attention mask. For causal attention, $\mathcal{M}_{ij} = 0$ if $j \le i$ and $\mathcal{M}_{ij} = -\infty$ otherwise. For full attention, $\mathcal{M}$ is all zeros.

2.2 Stacking Transformers Along the Depth Stream

Deep neural networks have enabled breakthroughs across domains, especially after the introduction of residual connections [16]. Scaling studies [18, 19, 21] further show that increasing depth can substantially improve performance [33, 36]. This motivates a natural question:

Is the residual connection the optimal mechanism for propagating information through depth stream?

Along the depth stream, we can view a Transformer block as a three-step procedure: **read, operate, and write**. We use this lens to describe different mechanisms for stacking Transformer blocks. For clarity, the first two mechanisms (Depth Residual [16], Depth Dense [20, 28]) are reference designs used to define the depth-stream design space. We introduce Depth Attention as an intermediate formulation and conceptual bridge. Our major technical contribution in this section starts from **Mixture-of-Depths Attention** (MoDA), which unifies sequence and depth retrieval in one unified softmax operator.

Depth Residual. In depth residual connections [16, 35], the "read" step is identity and the "write" step is add. The "operate" step is the token-mixing operator, i.e., attention, or the feed-forward network (FFN), denoted by $\mathcal{F}(\cdot)$. As shown in Fig. 3(a), the structure of depth residual can be formulated as follows:

$$X_l = X_0 + \sum_{i=1}^{l-1} \mathcal{F}(X_i, \mathcal{W}_i), \tag{3}$$

where $\mathcal{W}_i$ is the set of trainable weight matrices for the i-th layer.

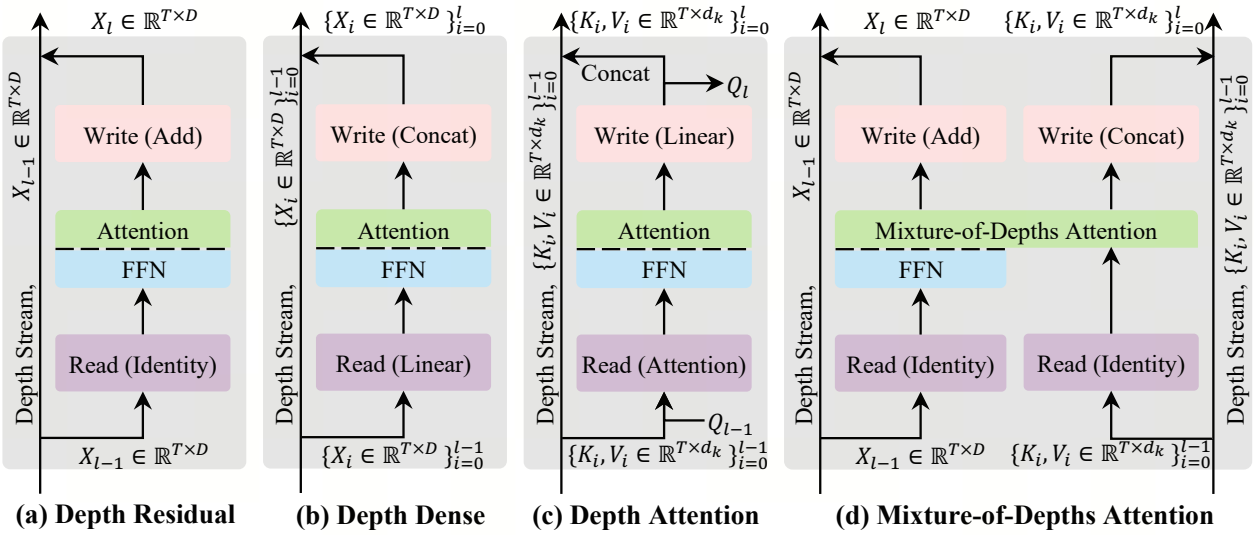

Figure 3 Conceptual comparison of mechanisms that utilize the depth stream. (a) **Depth Residual** [16] is the standard residual connection along depth: it reads the current representation and writes back by addition. (b) **Depth Dense** [20, 28] reads a set of historical representations and linearly projects them back to width D; it writes back by concatenation along depth, preserving all intermediate states. (c) We introduce **Depth Attention** as an intermediate formulation, which uses attention to read historical depth KV pairs in a data-dependent way. It writes back by concatenating the current layer's keys and values along depth. (d) We propose the upgraded version of Depth Attention, i.e., **Mixture-of-Depths Attention** (MoDA), which combines depth attention with standard sequence attention. It writes both the current layer's output and its KV pairs to depth streams for subsequent layers.

This formulation alleviates vanishing gradients and enables training deep networks. However, the depth stream is continuously *compressed* into a fixed-size tensor $X_l \in \mathbb{R}^{T\times D}$ via repeated superposition, which dilutes salient features and leads to signal degradation.

Depth Dense. To mitigate signal degradation, depth-dense methods [20, 28] connect all layers along the depth stream. At the "read" step, they form the input to layer l by linearly projecting the set of preceding representations $\{X_i \in \mathbb{R}^{T\times D}\}_{i=0}^{l-1}$ back to shape $T \times D$. At the "write" step, the layer output is concatenated with the historical set along depth. As shown in Fig. 3(b), the structure of depth dense can be formulated as follows:

$$\{X_i\}_{i=0}^{l} = \{X_0, \mathcal{F}(\{X_0\}, \mathcal{W}_1), \mathcal{F}(\{X_0, X_1\}, \mathcal{W}_2), \cdots, \mathcal{F}(\{X_i\}_{i=0}^{l-1}, \mathcal{W}_l)\}, \tag{4}$$

where $\mathcal{W}_i$ is the set of trainable weight matrices for the i-th layer.

Depth-dense connections propagate information through depth *losslessly*, because concatenation does not compress the historical set. However, they incur high cost and enforce a fixed connectivity pattern: the computation grows as $O(TL^2D^2)$ in dominant terms, which is prohibitive for large models.

Depth Attention. To reduce cost while retaining adaptive connectivity, we propose *depth attention* that reads historical depth information using attention in a data-dependent way, as illustrated in Fig. 3(c). At the "read" step, in the GQA-group view ($H_k d = D/G$), we denote one query-group representation by $Q_{l-1} \in \mathbb{R}^{T\times \frac{D}{G}}$ and the corresponding historical key-value sets by $\{K_i \in \mathbb{R}^{T\times \frac{D}{G}}\}_{i=0}^{l-1}$ and $\{V_i \in \mathbb{R}^{T\times \frac{D}{G}}\}_{i=0}^{l-1}$. The resulting input X_l^{in} is then fed into the "operate" step:

$$X_l^{\text{in}} = \text{Attention}(Q_{l-1}, \{K_i\}_{i=0}^{l-1}, \{V_i\}_{i=0}^{l-1}), \tag{5}$$

where attention is performed along the *depth* dimension: for token t, the query $Q_{l-1,t}$ attends only to the depth keys and values $\{K_{i,t}, V_{i,t}\}_{i=0}^{l-1}$ from the *same* token position across layers. After the "operate" step, the current layer output X_l^{out} is fed to the "write" step, which produces new query/key/value projections:

$$Q_l = X_l^{\text{out}} W_{Q,l}^{\text{W}}, \qquad K_l = X_l^{\text{out}} W_{K,l}^{\text{W}}, \qquad V_l = X_l^{\text{out}} W_{V,l}^{\text{W}}, \tag{6}$$

Table 1 Asymptotic complexity of depth-stream mechanisms. Here, T is the sequence length, D is the model width, G is the group size of Group Query Attention (GQA) [2], H_k is the number of key heads (equal to value heads H_v), H_q is the number of query heads ($=GH_k$), d is the head dimension, and L is the number of layers. We report dominant terms and omit constant factors.

Methods	Depth Dense	Depth Attention	Mixture-of-Depths Attention
Is data-dependent?	✘	✔	✔
Is unified softmax?	✘	✘	✔
Parameters	$\frac{1}{2}G^2L^2H_k^2d^2 + \frac{1}{2}G^2LH_k^2d^2$ $O(L^2D^2)$	$G^2LH_k^2d^2 + 2GLH_k^2d^2$ $O(LD^2)$	$2GLH_k^2d^2$ $O(L\frac{D^2}{G})$
Decoding Cache	GLH_kd $O(LD)$	$2LH_kd$ $O(L\frac{D}{G})$	$2LH_kd$ $O(L\frac{D}{G})$
Prefilling Cache	$GTLH_kd$ $O(TLD)$	$2TLH_kd$ $O(TL\frac{D}{G})$	$2TLH_kd$ $O(TL\frac{D}{G})$
Decoding FLOPs	$G^2LH_k^2d^2 + G^2L^2H_k^2d^2$ $O(L^2D^2)$	$2GL^2H_kd + 2GLH_kd$ $O(L^2D)$	$2GL^2H_kd + 2GLH_kd$ $O(L^2D)$
Prefilling FLOPs	$G^2TLH_k^2d^2 + G^2TL^2H_k^2d^2$ $O(TL^2D^2)$	$2GTL^2H_kd + 2GTLH_kd$ $O(TL^2D)$	$2GTL^2H_kd + 2GTLH_kd$ $O(TL^2D)$

where $W^{\mathrm{W}}_{Q,l}, W^{\mathrm{W}}_{K,l}, W^{\mathrm{W}}_{V,l} \in \mathbb{R}^{D\times\frac{D}{G}}$ are trainable matrices for the layer-l "write" operation, and $Q_l, K_l, V_l \in \mathbb{R}^{T\times\frac{D}{G}}$ denote per-group projections. We concatenate K_l and V_l along depth for future reads, while Q_l is passed forward to the next layer.

Compared with depth-dense connections, depth attention reads historical information adaptively with much lower cost. Its computation scales as $O(TL^2D)$, which is a factor of $\frac{1}{D}$ smaller than depth dense.

Mixture-of-Depths Attention. Building upon the Depth Attention, we now propose *mixture-of-depths attention* (MoDA). MoDA adds depth-level information to standard sequence-level attention and fuses these operations into a single operator. As illustrated in Fig. 1 and Fig. 3(d), MoDA reads the current hidden state X_{l-1} and the historical depth KV stream $\{(K_i, V_i)\}_{i=0}^{l-1}$. During the "operate" step, we apply MoDA to enable each token to attend to both the sequence-level keys and values and its own historical depth-wise keys and values, with all attention scores normalized jointly under a single softmax function. The implementation detail of MoDA is presented in Alg. 1. At the "write" step, for the attention layer, we append the current layer's key-value pair to the depth stream so that subsequent layers can access them. For the FFN layer, we obtain its corresponding key-value pair via a light-weight KV projection.

Overall, MoDA provides an efficient, data-dependent mechanism for exploiting depth history with substantially lower overhead than dense cross-layer connectivity. Furthermore, aggregating the sequence and depth information in one softmax operation provides a uniform representation space.

Complexity analysis. Complexity analysis is critical for modern LLM design, we also present the detailed complexity analysis among depth-aware designs, e.g., depth dense, depth attention, and MoDA. Table 1 reports complete complexity and dominant asymptotic terms, where T is sequence length, D is model width, L is the number of layers, head dimension d, and G is the GQA group size. Notably, $H_q = GH_k$.

From Table 1, Depth Dense is dominated by quadratic depth growth. Its parameter term is $O(L^2D^2)$, decoding cache is $O(LD)$, and both decoding and prefilling FLOPs contain quadratic-depth and quadratic-width terms, i.e., $O(L^2D^2)$ and $O(TL^2D^2)$. The proposed Depth Attention is a data-dependent method, which removes the dominant quadratic-width projection accumulation across depth, reducing parameters to $O(LD^2)$. It also lowers cache to $O(LD/G)$ and compute to $O(L^2D)$ and $O(TL^2D)$ for decoding and prefilling, respectively. Compared with Depth Attention, MoDA keeps the same favorable FLOPs order and cache order, but further reduces parameter complexity from $O(LD^2)$ to $O(LD^2/G)$. The key reason is that MoDA reuses the query projection from sequence attention, so no extra depth-query projection is introduced. Especially in GQA settings, only grouped depth key/value projections are needed. This makes MoDA the most parameter-efficient

Algorithm 1 MoDA: Hardware-aware Forward Pass

Input: $\mathbf{Q} \in \mathbb{R}^{T_q \times (H_k d)}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T_{kv} \times (H_k d)}$, $\mathbf{K}^{\text{depth}}, \mathbf{V}^{\text{depth}} \in \mathbb{R}^{(T_{kv} L) \times (H_k d)}$, group number G
Output: $\mathbf{O} \in \mathbb{R}^{T_q \times (H_k d)}$
Partition $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into hardware-friendly query/key/value blocks
Ensure each query block aligns with G for correct base-time mapping
for each query block index b_q **do**
  Load $\mathbf{Q}_{[b_q]}$ from HBM to SRAM (on chip)
  Initialize on-chip states: $m \leftarrow -\infty,\ acc \leftarrow 0,\ o \leftarrow 0$
  For each query row index i_q in block b_q, compute base-time: $t_{base}(i_q) = \lfloor i_q / G \rfloor$
  Let $t_{base}^{start} = \min_{i_q \in b_q} t_{base}(i_q)$ and $t_{base}^{last} = \max_{i_q \in b_q} t_{base}(i_q)$
  Define $t_{base}^{end} = t_{base}^{last} + 1$ as the exclusive upper bound
  for sequence key block b_s with $b_s < t_{base}^{start}$ **do**
    Load $(\mathbf{K}_{[b_s]}, \mathbf{V}_{[b_s]})$ from HBM to SRAM
    On chip, compute $S = \frac{\mathbf{Q}_{[b_q]} \mathbf{K}_{[b_s]}^\top}{\sqrt{d}}$
    On chip, calculate $\textsc{OnlineSoftmaxUpdate}(m, acc, o, S, \mathbf{V}_{[b_s]})$, i.e., the next two lines:
    On chip, $m' = \max(m, \max S)$, $acc' = acc \cdot 2^{m-m'} + \sum 2^{S-m'},\ o' = o \cdot 2^{m-m'} + \sum 2^{S-m'} \mathbf{V}_{[b_s]}$
    On chip, update $(m, acc, o) \leftarrow (m', acc', o')$
  end for
  for sequence key block b_s with $t_{base}^{start} \le b_s < t_{base}^{end}$ **do**
    Load $(\mathbf{K}_{[b_s]}, \mathbf{V}_{[b_s]})$ from HBM to SRAM
    Denote by i_k the sequence-key index in the current key block.
    On chip, compute $S = \frac{\mathbf{Q}_{[b_q]} \mathbf{K}_{[b_s]}^\top}{\sqrt{d}}$ and apply grouped causal mask $(\lfloor i_q / G \rfloor \ge i_k)$
    On chip, update $(m, acc, o) \leftarrow \textsc{OnlineSoftmaxUpdate}(m, acc, o, S, \mathbf{V}_{[b_s]})$
  end for
  for depth block index b_d with $t_{base}^{start} L \le b_d < t_{base}^{end} L$ **do**
    Load $(\mathbf{K}_{[b_d]}^{\text{depth}}, \mathbf{V}_{[b_d]}^{\text{depth}})$ from HBM to SRAM
    Denote by j_d the flattened depth-column index in the current block, i.e., $j_d \in \text{cols}(b_d)$.
    On chip, compute $S_d = \frac{\mathbf{Q}_{[b_q]} (\mathbf{K}_{[b_d]}^{\text{depth}})^\top}{\sqrt{d}}$ and apply $mask(i_q, j_d) = \mathbf{1}[\lfloor i_q / G \rfloor = \lfloor j_d / L \rfloor]$
    On chip, update $(m, acc, o) \leftarrow \textsc{OnlineSoftmaxUpdate}(m, acc, o, S_d, \mathbf{V}_{[b_d]}^{\text{depth}})$
  end for
  On chip, normalize $o \leftarrow o / acc$
  Store output block $\mathbf{O}_{[b_q]}$ from SRAM to HBM
end for
return O

option in Table 1, while preserving linear-in-width compute behavior and low-cache scaling.

Overall, Table 1 shows that MoDA keeps the data-dependent behavior of attention while avoiding the dominant quadratic-depth parameter growth overhead of dense cross-layer connections. MoDA aggregates sequence and depth information with a unified softmax operator, which provides better representation and efficiency in practice, especially in regimes with large L and long T.

3 Hardware-aware efficient MoDA

Naïvely PyTorch-implemented [29] MoDA requires non-contiguous reads of historical depth states, which degrades GPU utilization. We develop a hardware-aware implementation that reorganizes depth-stream tensors to enable contiguous memory access and fused computation.

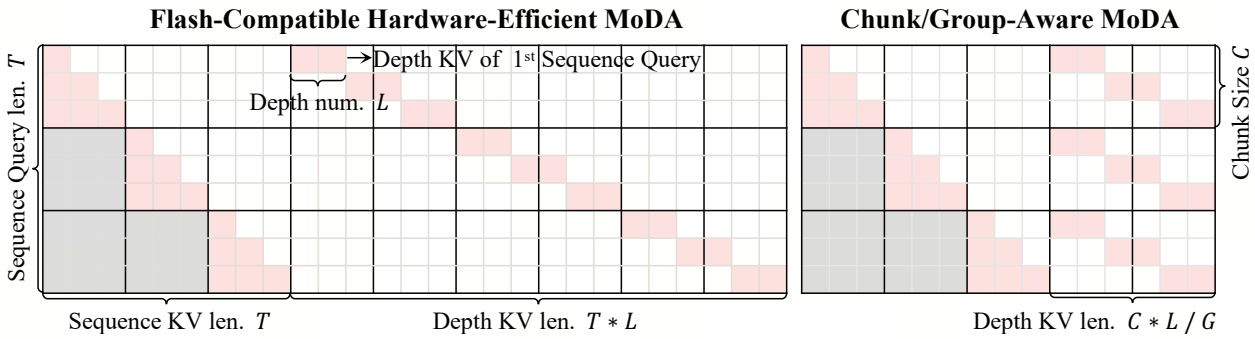

Figure 4 Hardware view of MoDA depth-cache access. **Left:** flash-compatible hardware-efficient MoDA keeps a depth KV cache of length $T \times L$ for each sequence, so each query potentially scans a long concatenated depth KV. **Right:** chunk-aware MoDA groups queries by chunk size C and reorganizes depth KV by chunk, reducing the effective depth span from $T \times L$ to $(C \times L)/G$ per chunk, where G is the GQA group number. This layout improves depth KV calculation efficiency and reduces memory access overhead.

3.1 Preliminary

Modern GPUs are optimized for throughput-oriented, large-scale data-parallel workloads, where the same operation is applied to many elements in parallel [12, 13, 44–46]. Therefore, efficient attention kernels should be organized to expose regular, massively parallel computation rather than irregular per-element control flow.

Streaming multiprocessors (SMs). An NVIDIA GPU is composed of many SMs, which are the basic on-chip units for parallel execution and resource management. High utilization requires enough independent blocks to keep many SMs active. In large language model (LLM) training with long-context sequences and relatively small batch sizes, parallelization along the temporal dimension is especially important.

Compute units: CUDA cores vs. Tensor Cores. Within each SM, instructions are dispatched to different execution units. CUDA cores support general arithmetic instructions, while Tensor Cores provide much higher throughput for structured matrix multiply-accumulate operations. As a result, practical high-performance kernels should maximize regular matmul-style computation to better exploit Tensor Cores.

Memory hierarchy: HBM and on-chip SRAM. End-to-end performance is jointly determined by compute throughput and data movement. HBM offers large capacity but higher access latency, whereas on-chip SRAM structures, i.e., registers, shared memory, and cache, are much faster but limited in size. Hence, a key design principle is to improve tiling and data reuse so that hot data stays on chip and HBM traffic is minimized.

These principles directly motivate our hardware-aware MoDA design. We reorganize depth KV layout and fuse computation to reduce non-contiguous memory access and improve effective compute utilization.

3.2 Hardware-aware Considerations for MoDA

Flash-Compatible depth KV layout. Naïvely implementing depth attention with explicit PyTorch for-loops over historical depth KV is typically slow on GPUs, because it induces irregular gather-like memory access and under-utilizes tensor-core-friendly block compute. Our first step is a flash-compatible depth-KV layout that flattens the depth cache along a single axis of length $T \times L$. Thus for each sequence position t, its L depth states are stored contiguously. In this way, each query only needs to map to its corresponding depth range $[tL, (t+1)L)$ to access the correct depth KV slice. This turns depth lookup into contiguous block reads and makes the depth phase compatible with FlashAttention-style kernels. Although this flattened formulation is substantially faster than explicit PyTorch for-loops over historical depth KV, it still introduces a compute-efficiency issue in the depth phase. In the depth-score matrix $\mathbf{S}^{\text{depth}} \in \mathbb{R}^{T \times (TL)}$, only a block-diagonal region is valid. Specifically, for query row i_q, only depth-column indices $j_d \in [i_q L, (i_q+1)L)$ are needed, while

the remaining entries are masked. We define this ratio as depth utilization, i.e., if computed densely over the full $T \times (TL)$ matrix, the depth utilization is $\eta_{\text{depth}} = \frac{T \cdot L}{T \cdot (T \cdot L)} = \frac{1}{T}$.

Chunk-aware depth KV layout. As illustrated in Fig. 4, flash-compatible depth KV layout forces each query block to traverse a long vectorized concatenated depth axis of length $T \times L$, which is unfavorable for depth utilization. We therefore reorganize depth KV in a chunk-aware manner, i.e., queries are divided into chunks, and each chunk only accesses the corresponding depth-KV span for its covered range. From a chunk-aware perspective, a query chunk of length C is paired with a local depth-KV region of size $C \times L$, constructed by concatenating the L depth states of the covered C sequence positions. The kernel therefore computes chunked depth attention over this packed $C \times L$ region, rather than scanning the global $T \times L$ depth axis for every chunk. This local layout substantially reduces unnecessary HBM traffic from masked, out-of-range depth entries and improves depth utilization to $\eta_{\text{depth}} = \frac{T \cdot L}{T \cdot (C \cdot L)} = \frac{1}{C}$.

Group-aware depth KV calculation. Our key observation is that, under the mapping $T_q = G\,T_{kv}$, G adjacent query rows share the same base-time index $\lfloor i_q/G \rfloor$ and can therefore reuse the same depth KV blocks. Based on this, we design a group-aware depth-KV computation, i.e., for a query chunk of length C, only C/G base-time rows are unique, so the required depth span is $(C/G) \times L$ rather than $C \times L$. Under the fused block-matmul and mask execution, this increases effective depth utilization to $\frac{G \times L}{C \times L} = \frac{G}{C}$. The same base-time mapping is used consistently in both masks, i.e., $\lfloor i_q/G \rfloor \geq i_k$ for sequence causality and $\lfloor i_q/G \rfloor = \lfloor j_d/L \rfloor$ for depth matching. Notably, i_k is the sequence-key index, while j_d is the flattened depth-column index. In practice, we also align query-block boundaries with G, i.e., make block size divisible by G, to avoid cross-group boundary handling inside one tile and simplify vectorized execution.

3.3 Hardware-Efficient MoDA Implementation

Preparation. Algorithm 1 follows the group-aware mapping $T_q = G\,T_{kv}$. The inputs are query $\mathbf{Q} \in \mathbb{R}^{T_q \times (H_k d)}$, sequence key/value $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T_{kv} \times (H_k d)}$, and depth key/value $\mathbf{K}^{\text{depth}}, \mathbf{V}^{\text{depth}} \in \mathbb{R}^{(T_{kv} L) \times (H_k d)}$, with output $\mathbf{O} \in \mathbb{R}^{T_q \times (H_k d)}$ and $H_k d = D/G$. For notation clarity, b_q, b_s, b_d denote block indices, while i_q, i_k, j_d denote element indices inside a block.

Before entering the main loops, all tensors are tiled into hardware-friendly blocks, and each query block is aligned to G. For each query block b_q, we load $\mathbf{Q}_{[b_q]}$ from HBM to SRAM and initialize on-chip online-softmax states (m, acc, o), where m is the running maximum logit, acc is the running softmax normalizer, and o is the running unnormalized output accumulator. For each query row index i_q in b_q, we compute its base-time index $t_{base}(i_q) = \lfloor i_q/G \rfloor$, and define $t_{base}^{start} = \min_{i_q \in b_q} t_{base}(i_q)$ and $t_{base}^{end} = \max_{i_q \in b_q} t_{base}(i_q) + 1$. The half-open interval $[t_{base}^{start}, t_{base}^{end})$ is then reused by both sequence and depth loops, ensuring index consistency. For intuition, if $G = 4$ and one query block contains rows $i_q = 8, \ldots, 15$, then $t_{base}(i_q) \in \{2, 3\}$, hence $t_{base}^{start} = 2$ and $t_{base}^{end} = 4$.

Sequence attention loops. The sequence phase contains two loops and both reuse the same accumulator states (m, acc, o). For fully visible blocks ($b_s < t_{base}^{start}$), we load $(\mathbf{K}_{[b_s]}, \mathbf{V}_{[b_s]})$ from HBM to SRAM, compute $S = \mathbf{Q}_{[b_q]} \mathbf{K}_{[b_s]}^{\top} / \sqrt{d}$, and call OnlineSoftmaxUpdate. In this region, all keys are earlier than the current query base-time, so no causal mask is required. For boundary blocks ($t_{base}^{start} \leq b_s < t_{base}^{end}$), the same pipeline is used with grouped causal masking $\lfloor i_q/G \rfloor \geq i_k$. Hence, logits from multiple sequence blocks are accumulated into one online-softmax state without intermediate HBM materialization. This is equivalent to processing a longer concatenated key sequence while keeping computation blockwise.

Depth attention loop. After sequence accumulation, the kernel enters the depth loop with flattened depth indices $b_d \in [t_{base}^{start} L, t_{base}^{end} L)$. The factor L maps a base-time index to its contiguous depth span of length L. For each depth block, $(\mathbf{K}_{[b_d]}^{\text{depth}}, \mathbf{V}_{[b_d]}^{\text{depth}})$ is loaded from HBM to SRAM, and depth logits $S_d = \mathbf{Q}_{[b_q]} (\mathbf{K}_{[b_d]}^{\text{depth}})^{\top} / \sqrt{d}$ are computed. We then apply the depth mask

$$mask(i_q, j_d) = \mathbf{1}\left[\left\lfloor \frac{i_q}{G} \right\rfloor = \left\lfloor \frac{j_d}{L} \right\rfloor\right] := \begin{cases} 1, & j_d \in \left[L \left\lfloor \frac{i_q}{G} \right\rfloor,\ L\left(\left\lfloor \frac{i_q}{G} \right\rfloor + 1\right)\right), \\ 0, & \text{otherwise.} \end{cases}$$

Table 2 Efficiency comparison of hardware-efficient MoDA and FlashAttention-2 Triton kernels under "forward&backward" setting. We report runtime (ms), depth utilization (η_{depth}), and relative extra time across three scaling settings. Here, B denotes batch size, d denotes head dimension, and C denotes chunk size. We launch all experiments on A100 GPU with bfloat16 data type.

No.	T	G	H_q	H_k	L	FA2-triton (ms)	MoDA-triton (ms)	Depth Utilization (η_{depth})	Extra Time Percentage
Scaling Sequence Length T (B=1, d=64, C=64)									
(1)	4096	8	64	8	64	7.970	10.750	12.50%	25.86%
(2)	8192	8	64	8	64	28.700	35.427	12.50%	18.99%
(3)	16384	8	64	8	64	116.700	127.661	12.50%	8.59%
(4)	32768	8	64	8	64	459.854	480.914	12.50%	4.38%
(5)	65536	8	64	8	64	1831.668	1883.026	12.50%	2.73%
Scaling GQA Group Size G (B=1, d=64, C=64)									
(6)	16384	2	16	8	64	28.982	39.741	3.12%	27.07%
(7)	16384	4	32	8	64	58.071	68.939	6.25%	15.76%
(8)	16384	8	64	8	64	116.700	127.661	12.50%	8.59%
(9)	16384	16	128	8	64	233.700	244.900	25.00%	4.57%
(10)	16384	32	256	8	64	467.107	480.767	50.00%	2.84%
Scaling Model Depth L (B=1, d=64, C=64)									
(11)	16384	8	64	8	64	116.700	127.661	12.50%	8.59%
(12)	16384	8	64	8	128	116.700	138.224	12.50%	15.57%
(13)	16384	8	64	8	256	116.700	167.958	12.50%	30.52%

which keeps only depth entries matched to the same base-time index as the query row. The masked logits are then passed to OnlineSoftmaxUpdate, reusing the same (m, acc, o) states as the sequence phase. Finally, we normalize once on chip via $o \leftarrow o/acc$, write $\mathbf{O}_{[b_q]}$ back to HBM, and return $\mathbf{O}$ after all query blocks are processed.

3.3.1 Efficiency Comparison

Table 2 reports end-to-end "forward&backward" runtime of hardware-efficient MoDA against FlashAttention-2 Triton under controlled settings. We sweep sequence length T, GQA group size G, and model depth L while fixing the remaining factors in each block (B=1, d=64, C=64). Besides raw runtime (ms), we also report depth utilization and the relative extra time percentage of MoDA.

When scaling sequence length, i.e., let T increase from 4096 to 65536, with G=8, L=64, both kernels follow the expected growth trend, while the relative extra time percentage of MoDA consistently decreases from 25.86% to 2.73%. This indicates that as sequence computation becomes dominant, the additional depth path is increasingly amortized. When scaling group size G from 2 to 32 at fixed T=16384, depth utilization rises from 3.12% to 50.00%, and the extra time percentage drops from 27.07% to 2.84%.

In contrast, when scaling model depth at fixed T=16384 and G=8, FlashAttention-2 runtime remains constant 116.700 ms, while MoDA runtime increases from 127.661 to 167.958 ms. Accordingly, the extra time percentage rises from 8.59% to 30.52%, which is consistent with the fact that deeper depth streams introduce more depth-KV processing. Overall, the results show that the proposed implementation has predictable linearly scaling behavior and remains efficient in long-sequence, high-utilization regimes.

4 Experiment

In this section, we demonstrate the expressivity and efficiency of the proposed MoDA through the experiments on Large Language Model (LLM).

Table 3 Performance of different mixture-of-depths attention (MoDA) variants on the training set, C4 validation set, and downstream benchmarks. We train the 700M models on 400B tokens. For **MoDA** settings: **'Sequence KV'** means the each token only attends to the sequence keys/values, can be regarded as the vanilla attention mechanism. **'Depth KV'** means the each token attends to its depth keys/values. **'Extra FFN KV Proj.'** means further project the FFN's input X to the depth keys/values, which are then used in subsequent attention operations. **'Extra Attn KV Proj.'** means set individual depth key/value projections rather than reuse the original key/value projections of sequence attention. The width D, GQA group size G, sequence length T are set to 1024, 2, and 4096, respectively. We further report the parameters and FLOPs of the models.

Model	Layer	Mixture-of-Depths Attention (MoDA)				Params (M)	FLOPs (T)	Train PPL	C4 Val PPL	Downstream Average
		Sequence KV	Depth KV	Extra FFN KV Proj.	Extra Attn KV Proj.					
Baseline Models										
(1) OLMo2	36	✔				669.0	8.01	14.49	18.59	56.93
(2) OLMo2	38	✔				700.5	8.41	14.27	18.31	57.11
Our Models										
(3) Ours	36	✔	✔			669.0	8.02	14.08	18.48	58.10
(4) Ours	36	✔	✔	✔		705.7	8.33	13.90	18.21	58.87
(5) Ours	36	✔	✔	✔	✔	742.4	8.63	13.83	18.17	58.97

4.1 Experimental Setups

Model Architecture and Training Settings. We conduct main experiments on language models of different sizes: 700M, and 1.5B. Following the popular practice, we adopt group query attention (GQA) [2] for 700M and 1.5B models. We train them on the 400B-token-subsets of OLMo2 [27] dataset. All models are trained in bfloat16 (bf16) precision. The global batch size is set to 1024, and the context sequence length is set to 4096. More detailed training configurations, such as learning rate schedule, AdamW [25] optimizer, etc., are following the OLMo2 [27] implementation.

Evaluation Details. We evaluate the models on popular benchmarks, including PiQA [5], HellaSwag [48], WinoGrande [32], OpenBookQA [26], BoolQA [9], SciQA [3], COPA [31], MMLU [17], ARC-easy (ARC-E) and ARC-challenge (ARC-C) [10]. We further report the training perplexity (PPL), C4 validation perplexity (Val PPL), and per-domain validation perplexity on C4 [30], ICE [27], m2d2-s2orc [24], Pile [14], Wiki-text [27], and dolma [34] validation sets, which includes Books, Common Crawl, peS2o, Reddit, and Stack.

4.2 Main Results

4.2.1 MoDA Variants

We first compare the results of different mixture-of-depths attention (MoDA) variants on the 700M model size. All models use a scheduler that warms up to a maximum learning rate of 3e-4 in 2k training steps, then decay to 3e-5 following the cosine schedule. We present experimental results in Table 3. To provide a fair comparison, we supplement the vanilla attention mechanism (OLMo2) as a baseline (row 1). Because the extra FFN KV projection introduces additional parameters, we also report the more-parameter baseline (row 2) with two additional layers. These methods introduce a comparable number of parameters/FLOPs than the proposed MoDA models.

From Table 3, we can observe that: **(i) Depth KV significantly improves performance**. Our method (row 3) keeps the same number of parameters as the baseline (row 1), but insert each token's depth KV into the attention computation. Note that we directly reuse the preceding layer's sequence KV as the depth KV, which would not introduce additional projection parameters. With only 0.12% extra FLOPs, it improves 0.41 train PPL, 0.11 C4 validation PPL, and 1.17 downstream averaged metrics (row 1 vs. row 3). **(ii) FFN layers' depth KV matters**. Experiment in row 3 only considers treat the preceding attention layers' KV as the depth KV, which ignores the FFN layers. We further add additional KV projections to enhance the original FFN , which projects the FFN's input X to its corresponding depth keys/values. Comparing row 3

Table 4 Performance of the proposed **MoDA** models with varying model sizes on the downstream benchmarks. We train the 700M and 1.5B models on the 400B tokens of OLMo2 dataset. The width D, GQA group size G, sequence length T are set to 1024, 2, and 4096, respectively. We mark the best performance with the **bold** font.

Model	PIQA	Hella-Swag	Wino-Grade	OpenBook-QA	BoolQA	SciQ	ARC-E	ARC-C	COPA	MMLU	Average
700M Models											
(1) OLMo2	**73.72**	58.77	55.33	35.60	56.24	89.50	66.84	33.44	77.00	24.69	57.11
(2) Ours	73.39	**59.19**	**60.22**	**37.20**	**59.33**	**89.60**	**67.37**	**34.78**	**82.00**	**25.61**	**58.87**
1.5B Models											
(3) OLMo2	76.55	65.86	63.22	38.80	63.61	90.60	**72.98**	42.47	81.00	27.73	62.28
(4) Ours	**76.82**	**66.24**	**65.59**	**41.60**	**67.34**	**92.10**	72.81	**46.82**	**85.00**	**29.59**	**64.39**

Table 5 Per-domain validation perplexity of the proposed **MoDA** models with varying model sizes. We train the 700M and 1.5B models on the 400B tokens of OLMo2 dataset. The width D, GQA group size G, sequence length T are set to 1024, 2, and 4096, respectively. Lower perplexity indicates better performance and is marked with the **bold** font.

Model	C4	ICE	m2d2-s2orc	Pile	Wiki-text	Books	CC	peS2o	Reddit	Stack	Average
700M Models											
(1) OLMo2	18.32	17.43	24.37	9.53	12.26	16.78	20.53	9.17	23.84	3.93	15.61
(2) Ours	**18.29**	**17.24**	**23.64**	**9.48**	**12.06**	**16.58**	**20.52**	**9.14**	**23.75**	**3.90**	**15.46**
1.5B Models											
(3) OLMo2	16.16	15.37	21.10	8.45	10.41	14.19	18.13	8.19	21.21	3.57	13.67
(4) Ours	**15.97**	**15.08**	**20.92**	**8.33**	**10.16**	**13.95**	**17.88**	**8.09**	**20.85**	**3.52**	**13.47**

and row 4, we can observe that incorporating KV from FFN improves 0.18 train PPL, 0.27 C4 validation PPL, and 0.77 downstream averaged metrics. While comparing row 4 with more-parameter baseline (row 2), it improves 0.37 train PPL, 0.10 C4 validation PPL, and 1.76 downstream averaged metrics. Notably, row 4 has similar number of parameters/FLOPs as row 2, but achieves better performance, which demonstrates that FFN's depth information also contributes to the mixture-of-depths attention (MoDA). **(iii) Extra Attn KV Projection is overly saturated**. Based on the row 4, we further introduce additional depth KV projection, which specifically projects the attention layers' input X to the depth keys/values. Comparing row 4 and row 5, we can observe that incorporating Extra Attn KV Projection only improves 0.07 train PPL, 0.04 C4 validation PPL, and 0.10 downstream averaged metrics. However, this modification introduces a non-trivial overhead (from 705.7M to 742.4M parameters and from 8.33T to 8.63T FLOPs), indicating that the additional attention-side depth projection is close to saturation.

Overall, these experiments reveal a clear design principle for MoDA: injecting depth information is effective, but the gains are highly sensitive to where additional projections are introduced. In particular, reusing attention-side depth KV already provides strong improvements at almost no cost, while adding FFN-side depth KV yields the best accuracy-efficiency trade-off. By contrast, introducing extra attention KV projections brings only marginal gains with noticeable parameter/FLOPs overhead. Therefore, we adopt the setting in row 4 as the default MoDA variant in the following scaling up experiments (Section 4.2.2).

4.2.2 Scaling MoDA with Model Size

We study whether the gains of MoDA persist when scaling model size from 700M to 1.5B under the same training budget of 400B tokens. We report downstream benchmark results in Table 4 and domain-level validation perplexity in Table 5. From these two tables, we can observe that: **(i) MoDA provides stable average gains on downstream benchmarks across model scales.** For 700M models, row 1 v.s. row 2 in Table 4 improves the average from 57.11 to 58.87, which is +1.76. For 1.5B models, row 3 v.s. row 4 improves the average from 62.28 to 64.39, which is +2.11. **(ii) Downstream gains are broadly observed across commonsense, reasoning, and broad-knowledge tasks.** On commonsense and causal discrimination

Table 6 Layer-number analysis of MoDA under deeper (48-layer) and shallower (24-layer) model settings. We compare vanilla attention (OLMo2) and MoDA variants with different MoDA choices, under both pre-norm and post-norm configurations. Models are trained with the same data recipe, and we report parameter count, FLOPs, and FineWeb-Edu validation loss. Across both depth regimes, introducing Depth KV consistently improves validation loss, and adding Extra FFN KV Projection yields further gains at moderate compute overhead.

Model	Layer	Norm	Mixture-of-Depths Attention (MoDA)			Params (M)	FLOPs (G)	FineWeb-Edu Val Loss
			Sequence KV	Depth KV	Extra FFN KV Proj.			
Experiments with Deeper Models (48 Layers)								
(1) OLMo2	48	prenorm	✔			123.38	136.61	3.3800
(2) OLMo2	48	postnorm	✔			123.38	136.61	3.4062
(3) Ours	48	prenorm	✔	✔		123.38	137.89	3.3759
(4) Ours	48	postnorm	✔	✔		123.38	137.89	3.3653
(5) Ours	48	prenorm	✔	✔	✔	128.11	144.00	3.3656
(6) Ours	48	postnorm	✔	✔	✔	128.11	144.00	3.3484
Experiments with Shallower Models (24 Layers)								
(7) OLMo2	24	postnorm	✔			71.35	78.19	3.4740
(8) Ours	24	postnorm	✔	✔		71.35	78.51	3.4537
(9) Ours	24	postnorm	✔	✔	✔	73.72	81.24	3.4338

tasks, the gains on HellaSwag, WinoGrande, and COPA are +0.42, +4.89, and +5.00 at 700M, and +0.38, +2.37, and +4.00 at 1.5B. On science-oriented and harder reasoning tasks, the gains on OpenBookQA, ARC-C, and SciQ are +1.60, +1.34, and +0.10 at 700M, and +2.80, +4.35, and +1.50 at 1.5B. We also observe consistent gains on broad-knowledge benchmarks, including BoolQ with +3.09 and +3.73, and MMLU with +0.92 and +1.86, for 700M and 1.5B, respectively. **(iii) Validation perplexity gains are broad and consistent across domains.** In Table 5, row 1 v.s. row 2 at 700M lowers average PPL from 15.61 to 15.46 and improves all ten domains. The largest 700M reduction appears on m2d2-s2orc, where PPL decreases from 24.37 to 23.64. At 1.5B, row 3 v.s. row 4 lowers average PPL from 13.67 to 13.47 and also improves all ten domains. Notable 1.5B reductions appear on Reddit from 21.21 to 20.85, on ICE from 15.37 to 15.08, and on Wiki-text from 10.41 to 10.16.

Overall, the two tables provide consistent evidence from complementary evaluation views. Table 4 shows improvements on end-task performance, while Table 5 shows improved language modeling quality across diverse domains.

4.3 Analysis

4.3.1 Analyzing MoDA with Layer Number

To study whether MoDA remains effective under different depth budgets, we conduct layer-number experiments on small models using the FineWeb-Edu data pipeline. We reserve an additional held-out split from FineWeb-Edu for validation and report validation loss for all settings. Specifically, we evaluate both deeper models (48 layers) and shallower models (24 layers), and compare vanilla attention with MoDA variants under pre-norm/post-norm configurations. For all runs in this subsection, the model width is 384, the number of query heads is 6, and the number of key/value heads is 2.

From the layer-number results, we observe that: **(i) Depth KV consistently improves validation loss across different layer numbers.** For 48-layer models, adding Depth KV reduces loss from 3.3800 to 3.3759 in pre-norm setting (row 1 vs. row 3), and from 3.4062 to 3.3653 in post-norm setting (row 2 vs. row 4). For 24-layer models, adding Depth KV also reduces loss from 3.4740 to 3.4537 (row 7 vs. row 8). **(ii) In deeper models, post-norm benefits more from Depth KV than pre-norm.** At 48 layers, row 2 vs. row 4 gives a loss reduction of 0.0409 in post-norm, while row 1 vs. row 3 gives 0.0041 in pre-norm. This indicates that Depth KV has stronger optimization impact in the post-norm configuration for deeper stacks. **(iii) Extra FFN KV Projection provides additional gains on top of Depth KV.** For 48-layer models, adding Extra FFN KV Projection further reduces loss from 3.3759 to 3.3656 in pre-norm (row 3 vs. row 5), and from 3.3653 to 3.3484 in post-norm (row 4 vs. row 6). For 24-layer models, it further reduces loss from

Table 7 Ablation of the proposed kernel implementation strategies under a fixed configuration. Each row incrementally enables one optimization component: (1) naive PyTorch baseline, (2) Flash-compatible depth-KV layout, (3) Flash-compatible & chunk-aware depth-KV layout, and (4) Flash-compatible & chunk-aware & group-aware indexing. We report end-to-end "forward&backward" runtime in milliseconds (ms), where lower is better and the best performance is marked with the **bold** font. Experiments are run on a single A100 GPU with bfloat16 under fixed setting B=1, T=1024, G=8, H_q=64, H_k=8, d=64, L=64, C=64.

No.	Naive Torch	Flash-Compatible	Chunk-Aware	Group-Aware	Time (ms)
B=1, T=1024, G=8, H_q=64, H_k=8, d=64, L=64, C=64					
(1)	✔				2128.900
(2)		✔			13.102
(3)		✔	✔		6.286
(4)		✔	✔	✔	**1.460**

3.4537 to 3.4338 (row 8 vs. row 9). Overall, these results show that MoDA remains effective under layer scaling, and FFN-side depth information brings additional gains when compute budget allows.

4.3.2 Analyzing MoDA with Attention Visualization

To better understand how MoDA changes token interactions, we visualize attention heatmaps for the **700M** model trained on **400B tokens** (Fig. 5). Under the combined-softmax formulation, each query attends over the concatenated *Sequence KV* | *Depth KV* space (red dashed line indicates the boundary). Notably, the depth-KV part contains both attention KV and FFN KV.

From the heatmaps, we observe non-trivial and persistent attention mass on the depth-KV block, especially in middle and late layers. This indicates that the model actively retrieves cross-layer depth information instead of relying only on sequence-local context. We also find a complementary pattern: heads with sharper diagonal sequence attention still allocate part of probability to depth slots, while broader heads tend to rely more heavily on depth-KV entries.

Another important observation is that MoDA exhibits attention patterns that differ from the typical *attention sink* behavior observed in the visualized heads. Rather than collapsing a large fraction of probability mass onto a few fixed sink positions, the attention in these heads appears to be distributed more broadly across sequence and depth slots, including slots that may be relevant to the task.

This qualitative difference suggests that MoDA may alter how attention mass is allocated in long-context settings. In particular, the visualization indicates that some probability mass is redistributed away from fixed sink positions toward sequence/depth locations that potentially carry useful information. While these patterns are intriguing, their precise functional role remains unclear and warrants further investigation.

Overall, the visualization is consistent with the core intuition of MoDA: depth information can serve as a complementary retrieval channel to standard sequence attention. At the same time, the changed attention-sink pattern may point to additional mechanisms or insights beyond the original design motivation, which should be studied more carefully in future work.

4.3.3 Analyzing MoDA with Efficiency

To quantify the practical efficiency contribution of each kernel design, we perform an incremental ablation and report end-to-end "forward&backward" runtime in Table 7. All experiments are conducted on a single A100 GPU with bfloat16 under fixed setting B=1, T=1024, G=8, H_q=64, H_k=8, d=64, L=64, C=64. Noting that naive PyTorch implementation is not optimized for efficiency, we only report the comparison under a short sequence length, i.e., T=1024.

From Table 7, we observe that: **(i) Flash-compatible depth-KV layout already provides orders-of-magnitude acceleration over naive implementation.** Row 1 vs. row 2 reduces runtime from 2128.900 ms to 13.102 ms, i.e., about 162.5× faster. **(ii) Chunk-aware depth-KV layout further improves**

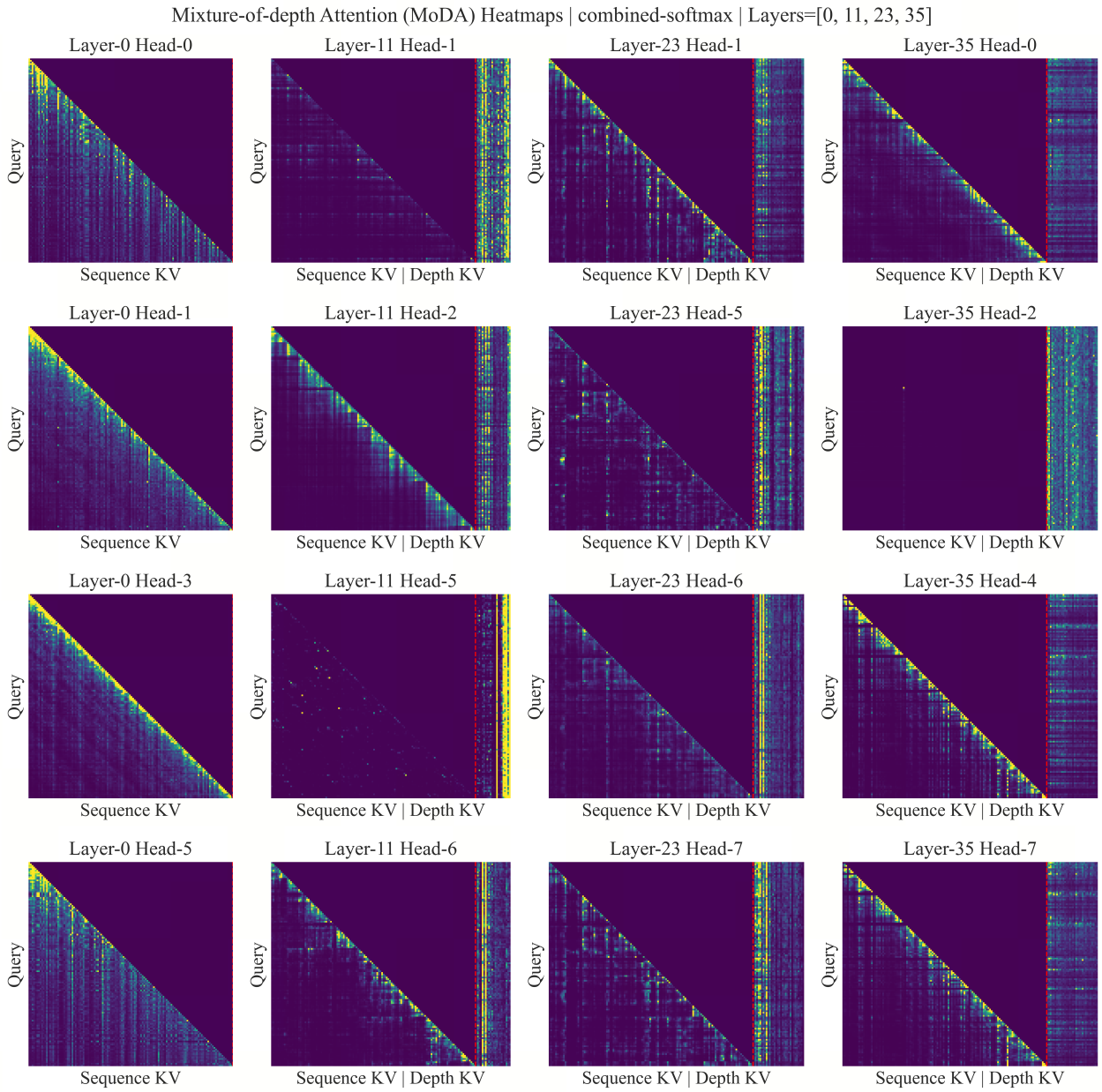

Figure 5 Mixture-of-Depths Attention (MoDA) heatmaps with the combined-softmax formulation. Columns correspond to uniformly sampled layers $\{0, 11, 23, 35\}$, and rows correspond to randomly selected heads in each layer. The first column shows attention over sequence KV only, while the other columns show concatenated *Sequence KV | Depth KV*; the red dashed line marks the boundary between the two KV blocks. Across layers and heads, substantial attention mass is consistently assigned to the depth-KV block, indicating that MoDA effectively leverages depth information in addition to standard sequence attention.

efficiency by reducing memory-access overhead. On top of Flash compatibility, row 2 vs. row 3 lowers runtime from 13.102 ms to 6.286 ms, corresponding to a 52.0% reduction. **(iii) Group-aware indexing is essential for fully exploiting group reusing mechanism.** Adding group-aware indexing (row 3 vs. row 4) further reduces runtime from 6.286 ms to 1.460 ms, giving an additional 4.31× speedup. Overall, combining all three optimizations yields the best runtime and achieves about 1458× end-to-end speedup over the naive PyTorch baseline (row 1 vs. row 4).

5 Conclusion

In this paper, we present MoDA, a unified depth-aware attention mechanism for LLM to improve depth-wise information aggregating and mitigate depth-efficiency gaps from optimization difficulty and information dilution. We further develop a hardware-aware fused kernel with unified online-softmax states, chunk-aware depth-KV layout, and group-aware indexing to maintain efficient long-context execution. Experiments on 700M and 1.5B models trained with the OLMo2 recipe show consistent gains in perplexity and downstream performance under modest overhead. These results suggest that explicit retrieval of historical depth information is a practical and effective primitive for scaling Transformer depth. We will release the full implementation of MoDA, and we hope it will serve as a foundation for building stronger large language models in the open-source community. Beyond language modeling, MoDA is architecture-agnostic and can be readily integrated into multimodal intelligence, visual understanding, and world models, where Transformers are increasingly adopted. We believe that principled depth-aware information aggregating will bring broad and lasting benefits across these diverse domains.

6 Discussion

6.1 Scaling MoDA for Industrial Training via Advanced CUDA Engineering

Although the current hardware-aware MoDA kernel already achieves competitive efficiency against FlashAttention-2, it is not yet the endpoint for industrial-scale training, e.g., trillion-parameter models. In large production runs, additional CUDA engineering remains critical, including improved memory scheduling, deeper computation pipelining, and tighter overlap between fused attention kernels and distributed communication. These optimizations do not change the algorithmic behavior of MoDA, but can further reduce memory stalls and kernel-launch overhead, improve end-to-end throughput, and increase cluster-level training efficiency. Therefore, we view future CUDA optimization as an important direction for turning MoDA from an efficient research operator into a robust primitive for industrial LLM training.

6.2 Mitigating Memory Bottlenecks with Bounded Depth-KV Slot Caching

When scaling to very deep networks, caching all depth-KV states from all historical layers introduces substantial memory and bandwidth overhead. The cost grows linearly with depth, and can become the dominant bottleneck in long-context training and serving. As a result, full depth-KV caching is increasingly hard to sustain at industrial scale.

A practical direction is to use a fixed-size Depth KV slot buffer. Instead of storing all depth-KV entries, each query only attends to a bounded set of slots. The slot budget is fixed to S, where $S \ll L$, and the system dynamically decides which depth-KV entries are kept. Two policies are natural. One is dynamic selection, which scores candidate depth-KV entries by utility and keeps the top-S entries. The other is a sliding-window policy, which keeps the most recent depth-KV entries and evicts older ones. A hybrid design can also be used, where part of the slots are reserved for recency and the rest for high-score global memories.

This design changes the effective depth memory from an unbounded cache to a bounded cache. The memory and bandwidth terms move from depth-dependent scaling to slot-dependent scaling. It also provides a stable tensor shape for fused kernel implementation. In practice, the key challenge is the quality of slot assignment. Future work should study how to train the selection policy jointly with MoDA, and how to balance quality, latency, and hardware efficiency under a fixed slot budget.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv:2303.08774*, 2023.

[2] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023.

[3] Sören Auer, Dante AC Barone, Cassiano Bartz, Eduardo G Cortes, Mohamad Yaser Jaradeh, Oliver Karras, Manolis Koubarakis, Dmitry Mouromtsev, Dmitrii Pliukhin, Daniil Radyush, et al. The sciqa scientific question answering benchmark for scholarly knowledge. *Scientific Reports*, 2023.

[4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv:2309.16609*, 2023.

[5] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *AAAI*, 2020.

[6] Chen Chen and Lai Wei. Post-layernorm is back: Stable, expressive, and deep. *arXiv:2601.19895*, 2026.

[7] Yunpeng Chen, Jianan Li, Huaxin Xiao, Xiaojie Jin, Shuicheng Yan, and Jiashi Feng. Dual path networks. *NeurIPS*, 2017.

[8] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv:1904.10509*, 2019.

[9] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *ACL*, 2019.

[10] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[11] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *ACL*, 2019.

[12] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*, 2023.

[13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *NeurIPS*, 2022.

[14] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv:2101.00027*, 2020.

[15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv:2501.12948*, 2025.

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv:2009.03300*, 2020.

[18] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically. *arXiv:1712.00409*, 2017.

[19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv:2203.15556*, 2022.

[20] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[21] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

[22] Baisheng Li, Banggu Wu, Bole Ma, Bowen Xiao, Chaoyi Zhang, Cheng Li, Chengyi Wang, Chengyin Xu, Chi Zhang, Chong Hu, et al. Virtual width networks. *arXiv:2511.11238*, 2025.

[23] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv:2405.04434*, 2024.

[24] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S Weld. S2orc: The semantic scholar open research corpus. In *ACL*, pages 4969–4983, 2020.

[25] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[26] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.

[27] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv:2501.00656*, 2024.

[28] Matteo Pagliardini, Amirkeivan Mohtashami, Francois Fleuret, and Martin Jaggi. Denseformer: Enhancing information flow in transformers via depth weighted averaging. *NeurIPS*, 2024.

[29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.

[30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 2020.

[31] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI*, 2011.

[32] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *CACM*, 2021.

[33] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*, 2014.

[34] Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, et al. Dolma: An open corpus of three trillion tokens for language model pretraining research. In *ACL*, 2024.

[35] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015.

[36] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.

[37] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv:2312.11805*, 2023.

[38] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv:2302.13971*, 2023.

[39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.

[40] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *TPAMI*, 2024.

[41] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv:2309.17453*, 2023.

[42] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, et al. mhc: Manifold-constrained hyper-connections. *arXiv:2512.24880*, 2025.

[43] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, 2024. URL https://github.com/fla-org/flash-linear-attention.

[44] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv:2312.06635*, 2023.

[45] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv:2412.06464*, 2024.

[46] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *NeurIPS*, 2024.

[47] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *ACL*, 2025.

[48] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *ACL*, 2019.

[49] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. *ICLR*, 2025.